```
  },
  "user": "anna@test.com",
  "SourceContext": "CityBreaks.Pages.CityModel",
```

While it is possible to write your own log-querying tools for working with structured logs, there are existing tools out there already, such as Seq (pronounced *seek*) and Elasticsearch, both of which Serilog includes sinks for. These third-party services may not be free; you should check their cost against the likely cost of developing and maintaining your own solution.

In this chapter, we've covered the main ways you can control how your application behaves in the event of errors occurring that you did not anticipate in code. We've also explored logging, primarily as a mechanism for capturing as much information about exceptions that get raised as possible, although it can be used to capture details of any kind of activity for any purpose. In the next chapter, we will look at protecting your application against external threats.

Summary

- During development, the Developer Exception page is displayed in the event of an unhandled exception, providing details of the exception and the line of code where it was raised.
- When the application is exposed to users, you should use `ExceptionHandlerMiddleware` to show a generic error page to hide details of errors.
- `ExceptionHandlerMiddleware` should be placed very early in the application pipeline.
- StatusCodePages middleware is designed to control how the application behaves when client errors occur.